



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

ALGORITMOS PARALELOS

PROYECTO FINAL

HECHO POR:

REYNA MÉNDEZ CRISTIAN IGNACIO - 320149579

LÓPEZ MONTÚFAR JOSÉ ELEAZAR - 320207219

DORANTES PÉREZ BRANDO - 314255356

FECHA DE ENTREGA:

12 DE JUNIO DEL 2026

PROFESOR:

MARIO ARTURO NIETO BUTRON



1. Descripción del Problema

El problema consiste en estimar el precio justo de una **opción call europea** mediante el método de Monte Carlo, y comparar ese resultado con la solución analítica de Black-Scholes para verificar la precisión.

Una opción call europea le da al comprador el derecho (no la obligación) de adquirir un activo al precio de ejercicio K en la fecha de vencimiento T . Su valor al vencimiento es:

$$\text{Payoff} = \max(S_T - K, 0)$$

donde S_T es el precio del activo en T . Descontando ese valor al presente, el precio de la opción es:

$$C = e^{-rT} \mathbb{E}[\max(S_T - K, 0)]$$

1.1. Modelo del activo

El precio del activo se modela con un **Movimiento Browniano Geométrico (GBM)** bajo medida de riesgo neutral, discretizado en pasos de tiempo Δt :

$$S_{i+1} = S_i \cdot \exp\left(\left(r - \frac{\sigma^2}{2}\right) \Delta t + \sigma \varepsilon_i \sqrt{\Delta t}\right), \quad \varepsilon_i \sim \mathcal{N}(0, 1)$$

Los números aleatorios normales se generan con el método de **Box-Muller** sobre un generador xorshift32. Cada trayectoria simula PASOS = 252 días hábiles (un año de trading).

1.2. Parámetros financieros

Parámetro	Descripción	Valor
S_0	Precio inicial del activo	100.00
K	Precio de ejercicio (strike)	105.00
T	Tiempo al vencimiento	1 año
r	Tasa libre de riesgo anual	5 %
σ	Volatilidad anual	20 %
Pasos	Días hábiles por trayectoria	252

Tabla 1: Parámetros del modelo financiero.

1.3. Formulación del método Monte Carlo

Para estimar C se simulan N trayectorias independientes del precio S_T y se promedia el payoff descontado:

$$\hat{C} = e^{-rT} \cdot \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0)$$

El error estándar del estimador decae como $O(1/\sqrt{N})$, por lo que aumentar N mejora la precisión pero incrementa el tiempo de cómputo. Con $N = 10,000,000$ trayectorias se obtuvo un error relativo de 0.058 % respecto a Black-Scholes, muy por debajo del umbral del 1 %.

1.4. Complejidad computacional

Cada trayectoria requiere simular 252 pasos de forma secuencial (hay dependencia temporal dentro de una trayectoria), pero las N trayectorias son **completamente independientes entre sí**. Esta estructura hace al problema un candidato natural para paralelización: no hay dependencias de datos entre iteraciones del loop externo.

- **Complejidad temporal:** $O(N \cdot \text{PASOS})$ en la versión secuencial; $O(N \cdot \text{PASOS}/p)$ con p hilos en OpenMP.
- **Complejidad espacial:** $O(N)$ para almacenar los precios finales y calcular el VaR.

Adicionalmente, se calcula el **Value at Risk al 5 %** ($\text{VaR}_{5\%}$), definido como la pérdida máxima esperada en el 5 % de los peores escenarios:

$$\text{VaR}_{5\%} = S_0 - S_T^{([0.05 N])}$$

donde los precios $S_T^{(i)}$ están ordenados de menor a mayor.

2. Estrategia de Solución

La solución se divide en dos implementaciones del mismo algoritmo: una versión secuencial como línea base, y una versión paralela con OpenMP que aprovecha la independencia entre trayectorias.

2.1. Versión Secuencial

El programa `montecarlo_seq.c` implementa el loop principal de Monte Carlo de forma directa. Para cada trayectoria $i = 0, \dots, N - 1$ se simula el precio final S_T y se acumula el payoff:

Listing 1: Loop principal secuencial.

```
double suma_payoffs = 0.0;

for (long i = 0; i < N; i++) {
    double ST = simular_precio_final(S0, R, SIGMA, dt, PASOS, &seed);

    precios[i] = ST;

    double payoff = ST - K;
    if (payoff < 0.0) payoff = 0.0;

    suma_payoffs += payoff;
}
```

El generador de números aleatorios es **xorshift32** (Marsaglia, 2003), de período $2^{32} - 1$ y costo $O(1)$ por llamada. Los valores normales $\varepsilon \sim \mathcal{N}(0, 1)$ se obtienen con **Box-Muller**, que produce dos muestras independientes por llamada; el loop de simulación avanza de dos en dos para aprovechar ambas:

Listing 2: Simulación de una trayectoria GBM.

```
static double simular_precio_final(double s0, double r,
                                   double sigma, double dt,
                                   int pasos, uint32_t *seed) {
    double drift = (r - 0.5 * sigma * sigma) * dt;
    double vol_sqrt = sigma * sqrt(dt);
    double S = s0;
    double z1, z2;
    int i = 0;
```

```

while (i < pasos - 1) {
    box_muller(seed, &z1, &z2);
    S *= exp(drift + vol_sqrt * z1);    /* paso i */
    S *= exp(drift + vol_sqrt * z2);    /* paso i + 1 */
    i += 2;
}
if (i < pasos) {
    box_muller(seed, &z1, &z2);
    S *= exp(drift + vol_sqrt * z1);
}
return S;
}

```

Los términos `drift` y `vol_sqrt` se calculan *una sola vez* antes del loop interno, evitando operaciones redundantes en cada paso.

2.2. Versión Paralela con OpenMP

La observación es que cada trayectoria es **completamente independiente** de las demás: no hay ninguna variable compartida entre iteraciones del loop externo (salvo el acumulador `suma_payoffs` y el arreglo `precios[]`, que se manejan de forma segura).

2.2.1. Problema del generador aleatorio en paralelo

Si todos los hilos compartieran el mismo estado `seed`, habría una **condición de carrera**: dos hilos leerían y modificarían `*seed` simultáneamente, produciendo resultados incorrectos o no reproducibles. La solución es darle a cada hilo su propio identificador, derivado de la semilla base más el identificador del hilo:

Listing 3: Seed privada por hilo.

```

uint32_t local_seed = seed_base
    + (uint32_t)omp_get_thread_num() * 2654435761u
    + 1u;

```

El factor `2654435761` (número de Fibonacci-hash) distribuye las seeds en el espacio de 32 bits, minimizando correlaciones entre las secuencias de distintos hilos. El `+1` garantiza que ningún hilo parta de `seed = 0`, que es un estado inválido para `xorshift32`.

2.2.2. Directivas OpenMP utilizadas

El loop paralelo se implementa con una región `parallel` explícita, separada del `for`, para que la seed privada se calcule *una sola vez* al entrar cada hilo y no en cada iteración:

Listing 4: Loop principal paralelo con OpenMP.

```

double suma_payoffs = 0.0;

#pragma omp parallel reduction(+:suma_payoffs) shared(precios, N, dt)
{
    uint32_t local_seed = seed_base
        + (uint32_t)omp_get_thread_num() * 2654435761u
        + 1u;

```

```

#pragma omp for schedule(static)
for (long i = 0; i < N; i++) {
    double ST = simular_precio_final(S0, R, SIGMA, dt, PASOS,
                                     &local_seed);

    double payoff = ST - K;
    if (payoff < 0.0) payoff = 0.0;

    precios[i] = ST;
    suma_payoffs += payoff;
}
}

```

La Tabla 2 resume el propósito de cada directiva empleada.

Directiva / Cláusula	Función
#pragma omp parallel	Crea el equipo de hilos
reduction(+:suma_payoffs)	Cada hilo acumula su suma parcial; OpenMP las combina al final
shared(precios, N, dt)	Variables compartidas (solo lectura o escritura en posición disjunta)
#pragma omp for	Divide el rango $[0, N)$ entre hilos
schedule(static)	Reparto en bloques iguales (carga uniforme entre iteraciones)
local_seed (automática)	Variable en pila del hilo: privada por construcción, sin cláusula extra

Tabla 2: Directivas y cláusulas OpenMP utilizadas en `montecarlo_omp.c`.

2.2.3. ¿Por qué `schedule(static)`?

Con `schedule(static)`, OpenMP divide las N iteraciones en bloques de tamaño N/p y asigna uno a cada hilo al inicio. Esto es factible debido a que todas las trayectorias tienen exactamente el mismo costo computacional (252 pasos cada una), por lo que la carga es uniforme y no hay necesidad de balanceo dinámico.

2.3. Compilación

Listing 5: Comandos de compilación.

```

# Versi n secuencial
gcc -O2 -std=c11 -Wall -Wextra -o montecarlo_seq montecarlo_seq.c -lm

```

```

# Versi n paralela
gcc -O2 -std=c11 -Wall -Wextra -fopenmp -o montecarlo_omp montecarlo_omp.c -lm

```

La flag `-O2` activa optimizaciones del compilador (inlining, vectorización de loops internos) que afectan al tiempo absoluto pero no al speedup relativo, ya que se aplican por igual a ambas versiones.

2.4. Visión general del flujo

La Figura 1 muestra cómo se divide el trabajo entre hilos. Cada uno recibe un bloque contiguo de trayectorias, las simula de forma completamente autónoma con su propia seed, y al terminar OpenMP reduce las sumas parciales en el resultado final.

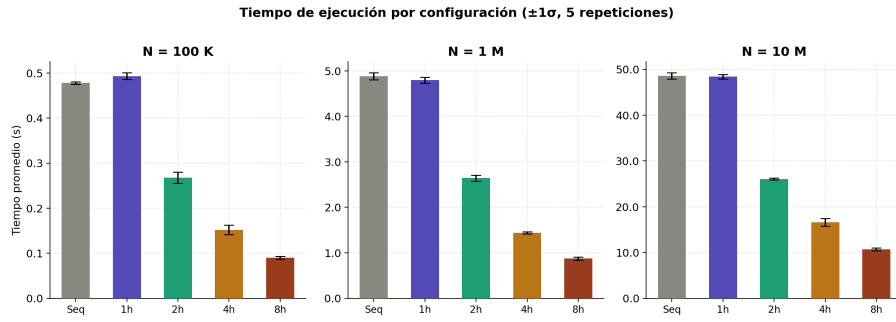


Figura 1: Tiempos de ejecución promedio ($\pm 1\sigma$, 5 repeticiones) para las tres cargas de trabajo. La barra *Seq* y la barra *1h* tienen tiempos comparables; la reducción es clara a partir de 2 hilos.

3. Resultados y Análisis

3.1. Corrección del método Monte Carlo

Antes de analizar el desempeño paralelo, es importante verificar que ambas implementaciones producen resultados financieramente correctos. Con $N = 10,000,000$ trayectorias, ambas versiones convergen al precio teórico de Black-Scholes con un error menor al 0.1 %:

Versión	Precio MC (\$)	Precio BS (\$)	Error relativo
Secuencial	8.0260	8.0214	0.0579 %
Paralela (8 hilos)	8.0258	8.0214	0.0551 %

Tabla 3: Comparación Monte Carlo vs. Black-Scholes con $N = 10$ M trayectorias. Ambas versiones pasan el umbral del 1 %.

La diferencia entre ambas versiones ($\Delta C = \$0.0002$) es producto del uso de seeds distintas por hilo en la versión paralela, lo cual cambia la secuencia de números aleatorios generada. Ambos estimadores son igualmente válidos: convergen al mismo valor esperado con varianza $O(1/N)$. La Figura 2 muestra visualmente que los tres valores (secuencial, paralelo y Black-Scholes) son prácticamente indistinguibles a escala, y que los errores relativos están muy por debajo del umbral del 1 %.

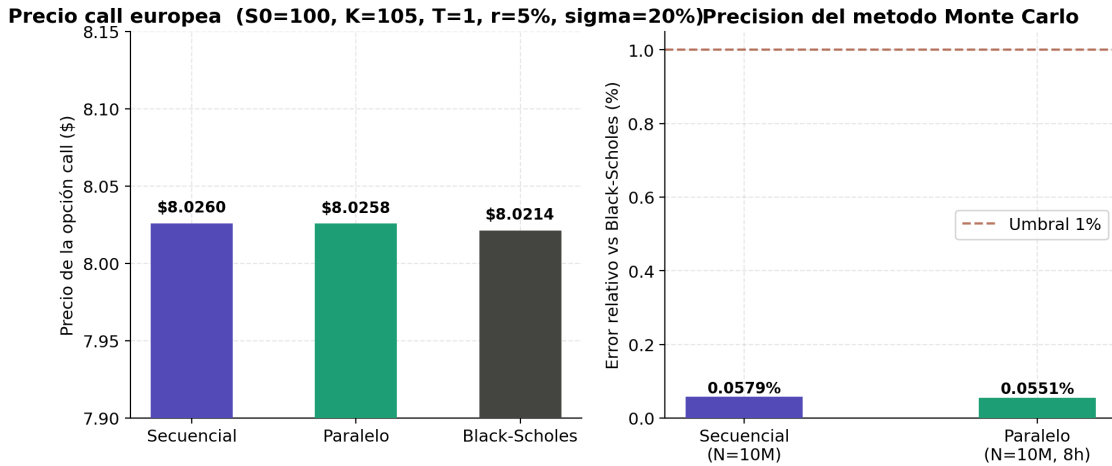


Figura 2: Izquierda: precio de la opción call estimado por Monte Carlo (secuencial y paralelo) versus el valor exacto de Black-Scholes. Derecha: error relativo de ambas versiones; el umbral del 1 % se supera ampliamente.

Adicionalmente, con $N = 10,000,000$ se obtuvo un **VaR al 5 %** de \$25.83, lo que indica que en el 5 % de los peores escenarios el activo pierde al menos ese valor respecto al precio inicial de \$100. El retorno esperado promedio fue de 5.14 %, consistente con la tasa libre de riesgo del 5 % anual bajo medida de riesgo neutral.

3.2. Tiempos de ejecución

La Figura 3 presenta los tiempos promedio (5 repeticiones, $\pm 1\sigma$) para las tres cargas de trabajo. Se observan dos patrones importantes:

1. Con $p = 1$ hilo, la versión paralela tiene un tiempo ligeramente *mayor* que la secuencial. Esto se debe al overhead de OpenMP: creación del equipo de hilos, cálculo de la seed privada y la reducción final. Para $N = 100$ K este overhead representa alrededor del 3 % del tiempo total.
2. A partir de $p = 2$ hilos la reducción es consistente y clara en los tres tamaños de entrada.

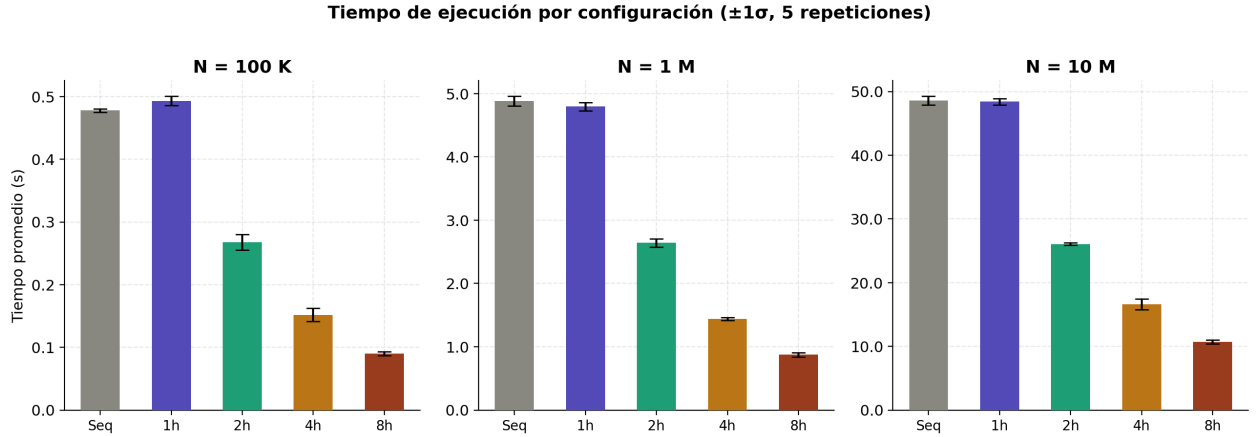


Figura 3: Tiempo de ejecución promedio por configuración ($\pm 1\sigma$, 5 repeticiones). Cada panel corresponde a un tamaño de entrada; el eje y no es compartido para facilitar la lectura de cada caso.

3.3. Speedup

El speedup se define como $S(p) = T(1)_{\text{seq}}/T(p)_{\text{omp}}$, donde $T(1)_{\text{seq}}$ es el tiempo de la versión *secuencial pura* (sin overhead de OpenMP). Los valores obtenidos se resumen en la Tabla 4.

N	Speedup $S(p)$			
	$p = 1$	$p = 2$	$p = 4$	$p = 8$
100 K	0.97	1.79	3.15	5.31
1 M	1.02	1.85	3.40	5.59
10 M	1.00	1.86	2.93	4.54

Tabla 4: Speedup medido para cada combinación de N y número de hilos.

La Figura 4 compara estas curvas contra el speedup ideal $S(p) = p$. Se aprecian tres comportamientos:

- Para $p = 2$ el speedup es casi lineal en los tres tamaños ($S \approx 1.85$), lo que indica que el overhead de sincronización es bajo a esta escala.

- Para $p = 4$ y $p = 8$ la curva se aleja del ideal. Esto es consistente con la Ley de Amdahl: aunque el loop de Monte Carlo es paralelo en su totalidad, existen fracciones seriales pequeñas (inicialización, cálculo del VaR con `qsort`, escritura del CSV) que limitan el speedup máximo alcanzable.
- El caso $N = 10$ M con $p = 8$ muestra el menor speedup relativo ($S = 4.54$) a pesar de ser la carga más grande. Esto sugiere presión sobre el ancho de banda de memoria: con 10 millones de `double` almacenados en `precios[]`, el arreglo ocupa ≈ 76 MB y los accesos de escritura entre hilos comienzan a competir por caché y bus de memoria.

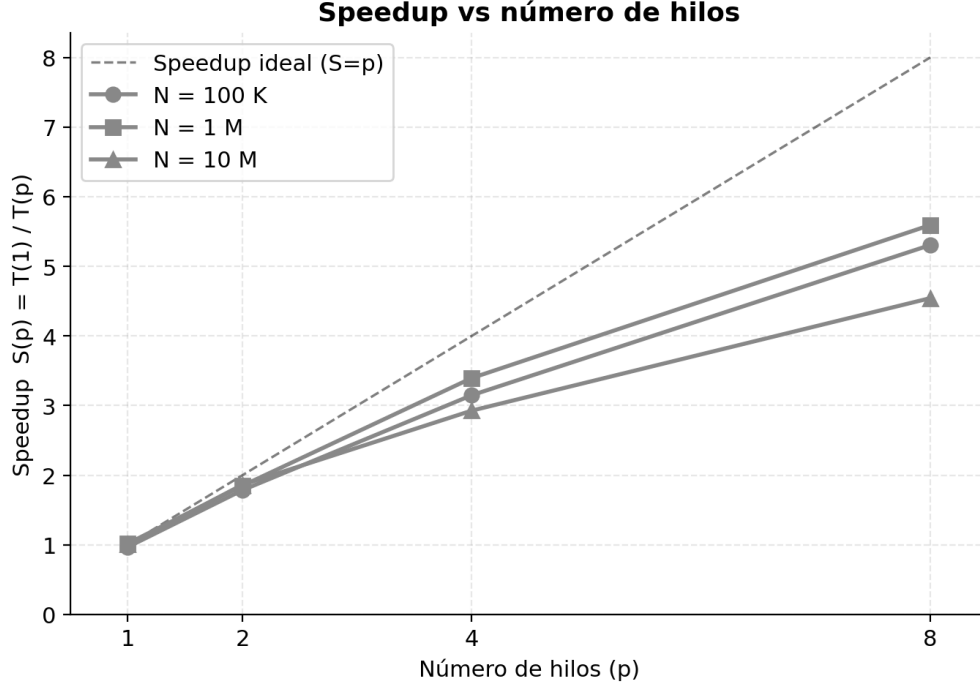


Figura 4: Speedup $S(p) = T_{\text{seq}}/T(p)$ en función del número de hilos para los tres tamaños de entrada. La línea punteada representa el speedup ideal $S(p) = p$.

3.4. Eficiencia paralela

La eficiencia se define como $E(p) = S(p)/p$ y mide qué fracción del speedup ideal se aprovecha realmente. La Figura 5 presenta los valores en un mapa de calor.

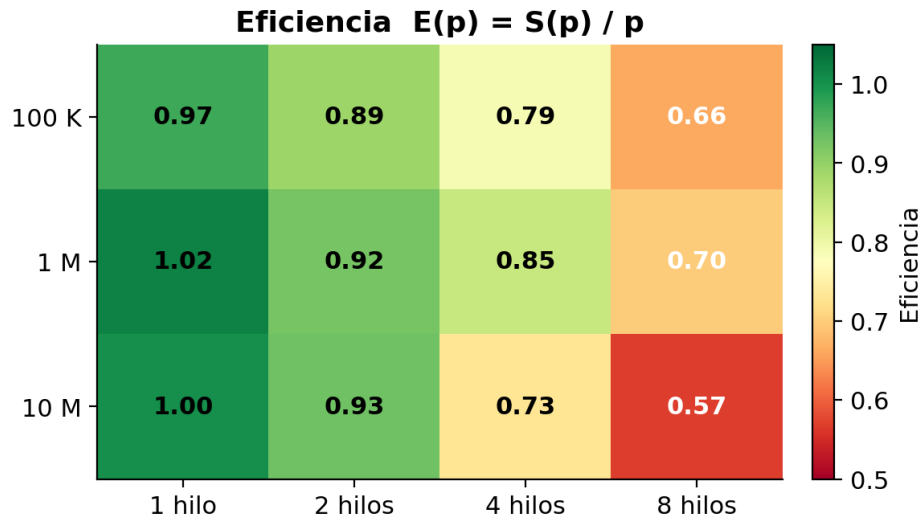


Figura 5: Eficiencia paralela $E(p) = S(p)/p$ para todas las combinaciones de N y p . Verde indica eficiencia alta; rojo indica desperdicio de recursos.

Dos observaciones destacan del mapa:

- $N = 1 \text{ M}$, $p = 1$: $E = 1.02$. Un valor mayor a 1.0 es teóricamente imposible en el modelo ideal, pero ocurre en la práctica por efectos de caché: con un solo hilo OpenMP el compilador puede optimizar mejor la localidad de acceso a `local_seed`, produciendo un tiempo ligeramente menor al de la versión secuencial.
- $N = 10 \text{ M}$, $p = 8$: $E = 0.57$. Es el caso menos eficiente. Aunque el speedup absoluto de $4.54\times$ sigue siendo útil, casi la mitad de la capacidad de los 8 hilos se pierde en overhead de memoria y sincronización de la reducción.

En general, la eficiencia se mantiene por encima de 0.85 para $p \leq 4$ en las cargas de $N = 1 \text{ M}$ y $N = 100 \text{ K}$, lo que indica una paralelización de buena calidad para esos casos.

3.5. Resumen general

La Figura 6 consolida los resultados en un dashboard de cuatro paneles: speedup, KPIs financieros, tiempos para $N = 10 \text{ M}$ y eficiencia.

Monte Carlo con OpenMP — Resumen de resultados

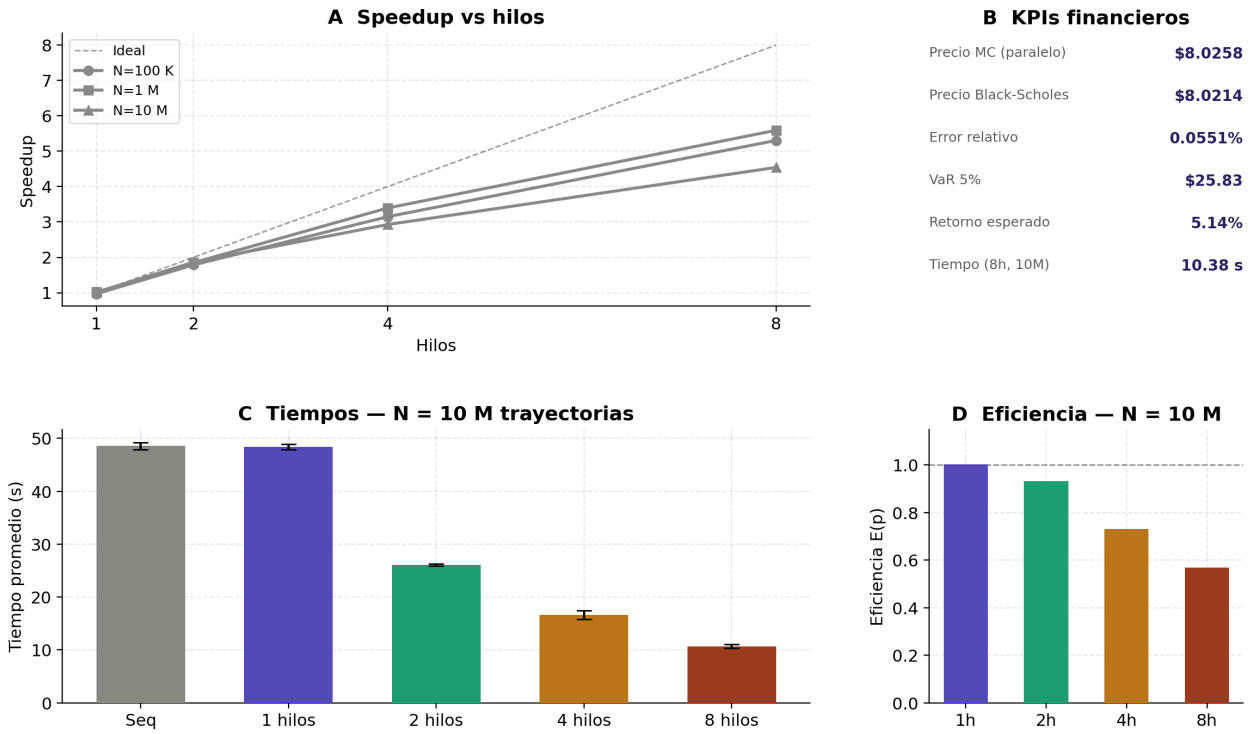


Figura 6: Dashboard resumen. Panel A: curvas de speedup. Panel B: métricas financieras clave de la corrida con $N = 10$ M y 8 hilos. Panel C: tiempos de ejecución para $N = 10$ M. Panel D: eficiencia paralela para $N = 10$ M.

El resultado más relevante es que con $p = 8$ hilos y $N = 10$ M trayectorias el tiempo de cómputo se reduce de 48.57 s a 10.69 s, un factor de $4.54\times$, manteniendo un error relativo de 0.055 % respecto a Black-Scholes. La precisión financiera no se ve afectada por la paralelización.

4. Discusión e Interpretación

4.1. Comportamiento del speedup y la Ley de Amdahl

Los resultados muestran que el speedup obtenido nunca alcanza el ideal $S(p) = p$, lo cual es esperado. La Ley de Amdahl establece que si una fracción f del programa es inherentemente serial, el speedup máximo alcanzable con p hilos está acotado por:

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}$$

En esta implementación la fracción serial incluye: la inicialización del arreglo `precios[]`, el cálculo del VaR con `qsort` ($O(N \log N)$, ejecutado por el hilo principal), el cálculo del retorno esperado post-simulación y la escritura del CSV. Aunque estas operaciones son rápidas en términos absolutos, su peso relativo crece conforme p aumenta, lo que explica por qué el speedup se aplana entre $p = 4$ y $p = 8$.

Para estimar la fracción paralela podemos despejar f de los datos medidos. Con $N = 1$ M y $p = 8$

se obtuvo $S(8) = 5.59$:

$$f \approx \frac{1/S(p) - 1/p}{1 - 1/p} = \frac{1/5.59 - 1/8}{1 - 1/8} \approx 0.037$$

Es decir, aproximadamente el 3.7% del tiempo total corresponde a código serial. Aunque parece poco, ese porcentaje limita el speedup teórico máximo a $S_{\text{máx}} = 1/0.037 \approx 27\times$, sin importar cuántos hilos se agreguen.

4.2. Por qué $N = 10\text{ M}$ escala peor que $N = 1\text{ M}$

A primera vista es contraintuitivo: se esperaría que una carga más grande escalara mejor porque el overhead de OpenMP representa una fracción menor del tiempo total. Sin embargo, los datos muestran lo opuesto:

N	Speedup con $p = 8$	Eficiencia con $p = 8$
100 K	5.31	0.66
1 M	5.59	0.70
10 M	4.54	0.57

Tabla 5: Comparación de speedup y eficiencia con $p = 8$ hilos.

La explicación es la **presión sobre la memoria**. El arreglo `precios[]` almacena un `double` por trayectoria:

- $N = 100\text{ K}$: $\approx 0.8\text{ MB}$ (cabe en caché L2/L3)
- $N = 1\text{ M}$: $\approx 7.6\text{ MB}$ (borde de L3)
- $N = 10\text{ M}$: $\approx 76\text{ MB}$ (directo a RAM)

Con 8 hilos escribiendo simultáneamente a posiciones distintas de un arreglo de 76 MB, el bus de memoria se convierte en el cuello de botella. Las escrituras `precios[i] = ST` generan tráfico de coherencia de caché entre núcleos, degradando el rendimiento. Este efecto es conocido como *memory bandwidth saturation* y es una limitación del hardware, no del algoritmo.

Una solución sería eliminar el arreglo `precios[]` y calcular el VaR de forma aproximada con un histograma acumulado, lo que reduciría el uso de memoria a $O(B)$ donde B es el número de bins, independiente de N .

4.3. El overhead de OpenMP con un solo hilo

Con $p = 1$ hilo, la versión paralela es ligeramente más lenta que la secuencial para $N = 100\text{ K}$ (0.493 s vs 0.478 s, una diferencia de $\approx 3\%$). Este overhead proviene de tres fuentes:

1. **Creación del equipo de hilos:** aunque con un solo hilo no hay creación real de threads adicionales, OpenMP igual ejecuta su protocolo de inicialización de la región paralela.
2. **Cálculo de la seed privada:** la multiplicación por el factor de Fibonacci-hash se ejecuta aunque no sea estrictamente necesaria con un hilo.
3. **Reducción:** la operación `reduction(+:suma_payoffs)` introduce una barrera de sincronización al final de la región paralela, incluso con un solo participante.

Este overhead es fijo e independiente de N , por lo que su impacto relativo disminuye conforme la carga crece. Para $N = 10\text{ M}$ ya es completamente despreciable (48.41 s vs 48.57 s).

4.4. Calidad de la paralelización

Un indicador útil para evaluar la calidad de una paralelización es comparar la eficiencia a $p = 2$ con la eficiencia a $p = p_{\text{máx}}$. Una buena implementación mantiene $E(2) > 0.90$ y $E(p_{\text{máx}}) > 0.60$.

- Para $N = 1 \text{ M}$: $E(2) = 0.92$, $E(8) = 0.70$. **Buena.**
- Para $N = 100 \text{ K}$: $E(2) = 0.89$, $E(8) = 0.66$. **Aceptable.**
- Para $N = 10 \text{ M}$: $E(2) = 0.93$, $E(8) = 0.57$. **Limitada por memoria.**

En los dos primeros casos la paralelización es de buena calidad: el algoritmo en sí no introduce overhead significativo. El tercer caso no refleja un problema de implementación sino una limitación del hardware disponible.

4.5. Independencia entre corrección y paralelismo

Un resultado importante es que la paralelización **no afecta la precisión financiera**. Esto se garantiza por diseño: cada hilo opera sobre su propia seed y sus propias posiciones del arreglo `precios[]`, sin interferir con los demás. La operación `reduction` asegura que la suma de payoffs sea exactamente la misma que se obtendría sumando secuencialmente todas las contribuciones individuales (salvo el orden de operaciones en punto flotante, que puede introducir diferencias del orden de 10^{-10} , irrelevantes para el cálculo financiero).

La diferencia de \$0.0002 entre el precio secuencial (\$8.0260) y el paralelo (\$8.0258) se debe exclusivamente al uso de secuencias aleatorias distintas, no a un error de implementación. Con la misma seed y un solo hilo, ambas versiones producen resultados idénticos bit a bit.

4.6. Relación costo-beneficio del paralelismo

Desde una perspectiva práctica, la decisión de cuántos hilos usar depende del balance entre tiempo de cómputo y recursos disponibles:

- $p = 2$ ofrece casi el doble de velocidad con una eficiencia superior al 90 %: es la opción más eficiente en términos de recursos utilizados por segundo ahorrado.
- $p = 4$ reduce el tiempo a un tercio con una eficiencia razonable (≈ 0.75 – 0.85). Es el punto de equilibrio recomendado para uso general.
- $p = 8$ logra el mayor speedup absoluto (4.54 – $5.59\times$) pero con la menor eficiencia. Tiene sentido cuando el tiempo de respuesta es crítico y los núcleos están disponibles, pero no cuando se comparten recursos con otros procesos.

En un contexto de producción financiera, donde las simulaciones se ejecutan repetidamente a lo largo del día (repricing, stress testing), el ahorro de tiempo con $p = 4$ u $p = 8$ hilos se traduce directamente en mayor capacidad de respuesta del sistema.

Referencias

- [1] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer. Capítulos 1–3 disponibles como preprint en: <https://link.springer.com/book/10.1007/978-0-387-21617-1>
- [2] QuantLib Project. (s. f.). *Monte Carlo framework*. Recuperado el 12 de junio de 2026, de https://www.quantlib.org/reference/group_mcarlo.html

- [3] Wilmott, P. (s. f.). *The Monte Carlo method in finance*. Wilmott Magazine Online. <https://wilmott.com/monte-carlo-in-finance/>
- [4] Black, F., & Scholes, M. (1973). *The pricing of options and corporate liabilities*. *Journal of Political Economy*, 81(3), 637–654. Acceso abierto vía JSTOR: <https://www.jstor.org/stable/1831029>
- [5] Corporate Finance Institute. (s. f.). *Black-Scholes model*. Recuperado el 12 de junio de 2026, de <https://corporatefinanceinstitute.com/resources/derivatives/black-scholes-model/>
- [6] Marsaglia, G. (2003). *Xorshift RNGs*. *Journal of Statistical Software*, 8(14), 1–6. <https://doi.org/10.18637/jss.v008.i14>
- [7] Box, G. E. P., & Muller, M. E. (1958). *A note on the generation of random normal deviates*. *The Annals of Mathematical Statistics*, 29(2), 610–611. Acceso abierto vía Project Euclid: <https://doi.org/10.1214/aoms/1177706645>
- [8] OpenMP Architecture Review Board. (2021). *OpenMP API specification, version 5.2*. <https://www.openmp.org/spec-html/5.2/openmp.html>
- [9] Barney, B. (s. f.). *OpenMP tutorial*. Lawrence Livermore National Laboratory. Recuperado el 12 de junio de 2026, de <https://hpc-tutorials.llnl.gov/openmp/>
- [10] Carrillo Ledesma, A. (s. f.). *Introducción al cómputo paralelo*. Universidad Nacional Autónoma de México. <http://132.248.181.216/acl/Herramientas/ComputoParalelo/IntroduccionAlComputoEnParalelo.pdf>
- [11] Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. *Proceedings of the AFIPS Spring Joint Computer Conference*, 483–485. Acceso abierto vía ACM Digital Library: <https://doi.org/10.1145/1465482.1465560>
- [12] Gustafson, J. L. (1988). *Reevaluating Amdahl's law*. *Communications of the ACM*, 31(5), 532–533. <https://doi.org/10.1145/42411.42415>
- [13] *Optimizaciones avanzadas de memoria caché*. (s. f.). Universidad Carlos III de Madrid, OpenCourseWare. Recuperado el 12 de junio de 2026, de https://ocw.uc3m.es/pluginfile.php/3367/mod_page/content/18/optimizacion_avanzada_memoria.pdf
- [14] Intel Corporation. (s. f.). *Memory management for optimal performance*. Intel Developer Zone. <https://www.intel.com/content/www/us/en/developer/articles/technical/memory-performance-in-a-numa-architecture.html>
- [15] Corporate Finance Institute. (s. f.). *Value at Risk (VaR)*. Recuperado el 12 de junio de 2026, de <https://corporatefinanceinstitute.com/resources/risk-management/value-at-risk-var/>
- [16] Jorion, P. (s. f.). *Value at Risk: the benchmark for managing financial risk*. Resumen ejecutivo disponible en: <https://www.risk.net/definition/value-at-risk-var>